

Contents

□ Dynamic Programming — Strings (LCS, Edit Distance, Palindromes) — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	5
SECTION 6 — Time & Space Complexity	6
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	8
SECTION 11 — Pattern Variations	8
SECTION 12 — Comparison With Other Patterns	9
SECTION 13 — Problem Classification	9
SECTION 14 — 30 Interview Problems	10
SECTION 15 — Common Mistakes	13
SECTION 16 — Optimization Techniques	13
SECTION 17 — Multiple Language Templates	14
SECTION 18 — Dry Runs	15
SECTION 19 — Advanced Concepts	16
SECTION 20 — Senior Engineer Insights	16
SECTION 21 — Cheat Sheet	16
SECTION 22 — Revision Notes (5-Minute Review)	17
SECTION 23 — Practice Roadmap	17
SECTION 24 — Memory Tricks	17
SECTION 25 — Final Summary	18

□ **Dynamic Programming — Strings (LCS, Edit Distance, Palindromes) — Complete Interview Handbook**

Pattern #25 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — DP on Strings section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary
-

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

DP on Strings is the specialized family of 2D DP (Pattern #23) applied specifically to **comparing, transforming, or analyzing one or two strings**. The two dominant sub-families are: **sequence comparison** (Longest Common Subsequence, Edit Distance — comparing two distinct strings character by character) and **palindrome analysis** (Longest Palindromic Subsequence/Substring — comparing a string against **itself**, typically via an interval-based $dp[i][j]$ representing the substring/subsequence from index i to j).

1.2 Why Was This Pattern Invented?

Comparing two strings for similarity (how many edits to transform one into another, or how much they share) or analyzing a string's internal symmetry (palindromic structure) both have exponentially many possible alignments/partitions if brute-forced. String DP exploits the same optimal-substructure/overlapping-subproblems insight as general 2D DP: the best alignment/partition of a longer prefix or substring is built from the best alignment/partition of a shorter one.

1.3 Real Intuition Behind The Pattern

Think of **aligning two strips of paper, one above the other, to maximize how many letters line up in the same order** (without needing to be adjacent) — that's LCS. Think of **counting the minimum number of single-letter edits (insert/delete/replace) to turn one word into another**, like a spell-checker's "did you mean" feature — that's Edit Distance. Think of **checking how far you can "peel" matching letters from both ends of a string inward** — that's palindrome DP.

1.4 Mental Model

For sequence comparison: "do the current characters of both strings match? If so, extend the best solution from one step back in both; if not, take the best of skipping a character from either string." For palindromes: "does the substring from i to j become a palindrome by matching its outer characters and having a palindromic (or empty/single-char) inner substring?"

1.5 Visual Explanation

LCS of "ABCBADAB" and "BDCABA":

	"	B	D	C	A	B	A
"	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

$dp[i][j] = dp[i-1][j-1] + 1$ if match, else $\max(dp[i-1][j], dp[i][j-1])$

Final: $dp[7][6] = 4 \rightarrow$ LCS length = 4 (e.g., "BCBA" or "BDAB")

Longest Palindromic Substring, interval DP on "babad":

$dp[i][j] = \text{true}$ if substring $i..j$ is a palindrome

Base: $dp[i][i] = \text{true}$ (single char); $dp[i][i+1] = (s[i] == s[i+1])$

$dp[i][j] = (s[i] == s[j]) \text{ AND } dp[i+1][j-1]$ - fill by increasing substring length

1.6 Simple Analogy

LCS is like **finding the longest sequence of matching street names you'd pass driving down two different (winding) roads**, in order but not necessarily consecutively. Edit Distance is like **counting spell-check corrections**. Palindrome DP is like **checking if peeling matching letters off both ends of a word, repeatedly, eventually empties it out (or leaves a single middle letter)**.

1.7 When Should I Immediately Think About Using This Pattern?

- “Longest common subsequence/substring” between two strings.
 - “Minimum edit distance,” “minimum operations to convert.”
 - “Longest palindromic substring/subsequence,” “minimum cuts to partition into palindromes.”
 - “Is one string a subsequence/anagram-transformable/interleaving of others.”
-

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“longest common subsequence/substring”	Sequence-comparison 2D DP
“edit distance,” “minimum operations to convert”	Edit Distance DP
“longest palindromic substring/subsequence”	Palindrome interval DP
“minimum cuts to partition into palindromes”	Palindrome DP + 1D DP combination
“distinct subsequences,” “interleaving string”	Advanced sequence-comparison DP

2.2 Hidden Hints

Any problem comparing **two strings** for similarity/transformation, or asking about a **single string's internal symmetric structure**, is a strong signal for this pattern family — even if “DP” isn't mentioned explicitly.

2.3 Interview Clues

Interviewer gives two strings and asks about their relationship (common parts, edit cost), or gives one string and asks about palindrome-related structure.

2.4 Common Trick Words

“Minimum operations,” “transform,” “convert,” “palindrome,” “common” — all point directly to this pattern family.

2.5 What Interviewers Expect

Correct recurrence design distinguishing match vs. mismatch cases, correct base-case handling for empty-prefix comparisons, and — for palindrome problems — recognizing the interval-DP fill order (by increasing substring length, not row-by-row) as distinct from standard grid DP.

2.6 When NOT To Use This Pattern

- You only need **existence** of a substring/pattern (not the longest/optimal one) — simpler string-matching algorithms (KMP, Rabin-Karp) may be more appropriate and faster.

- The comparison is between more than two strings simultaneously in a complex way — may need a generalized/higher-dimensional DP or a completely different technique (e.g., suffix trees/arrays for multi-string problems).
- You need **all** common subsequences enumerated, not just the longest one's length — that requires backtracking layered on top of the DP table.

SECTION 3 — Decision Framework

Are you comparing TWO DIFFERENT strings for similarity/transformation?

Yes → USE SEQUENCE-COMPARISON DP ($dp[i][j]$ over prefixes of both strings)

Match case: $dp[i-1][j-1] + 1$ (LCS) or $dp[i-1][j-1]$ (Edit Distance, no-op)

Mismatch case: max/min combination of $dp[i-1][j]$, $dp[i][j-1]$, ($dp[i-1][j-1]$ for Edit Distance)

No

Are you analyzing a SINGLE STRING'S internal palindrome/symmetric structure?

Yes → USE INTERVAL DP ($dp[i][j]$ over the substring from i to j , filled by increasing length)

No

Do you need just PATTERN EXISTENCE/MATCHING (not longest/optimal), possibly with wildcards?

Yes → Consider KMP/Rabin-Karp (existence) or DP for wildcard/regex matching (Pattern #23's regex/wildcard)

Why: The defining split within DP on Strings is whether you're comparing two independent strings (needs a full (i, j) grid over both strings' prefixes) or analyzing one string's internal structure (needs an interval (i, j) representing a substring range, filled in a fundamentally different order — by increasing length, not row-by-row).

SECTION 4 — Why This Pattern Works

LCS Correctness Proof: *State:* $dp[i][j]$ = length of LCS of $A[0..i-1]$ and $B[0..j-1]$. *Base case:* $dp[0][j] = dp[i][0] = 0$ (LCS with an empty string is always 0). *Inductive step:* if $A[i-1] == B[j-1]$, this matching character can always be included in an optimal LCS (a classical exchange argument: any LCS not using this match can be modified to use it without decreasing length), so $dp[i][j] = dp[i-1][j-1] + 1$. If they don't match, the LCS of $A[0..i-1]$ and $B[0..j-1]$ must exclude the last character of at least one string, so $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$, exhaustively covering both possibilities. *Termination:* $dp[m][n]$ is the correct LCS length by induction. **QED.**

Edit Distance Correctness Proof: *State:* $dp[i][j]$ = minimum operations to convert $A[0..i-1]$ into $B[0..j-1]$. *Base case:* $dp[i][0] = i$ (delete all of A), $dp[0][j] = j$ (insert all of B). *Inductive step:* if $A[i-1] == B[j-1]$, no operation is needed for this character, so $dp[i][j] = dp[i-1][j-1]$. Otherwise, the last operation converting A's prefix to B's prefix must be an insert ($dp[i][j-1]+1$), delete ($dp[i-1][j]+1$), or substitute ($dp[i-1][j-1]+1$) — taking the minimum of these three exhaustively covers every possible final operation. *Termination:* $dp[m][n]$ is correct by induction.

Palindrome Interval DP Correctness Proof: *State:* $dp[i][j]$ = true iff substring $s[i..j]$ is a palindrome. *Base case:* $dp[i][i] = \text{true}$ (single character); $dp[i][i+1] = (s[i] == s[i+1])$. *Inductive step:* $dp[i][j] = (s[i] == s[j]) \text{ AND } dp[i+1][j-1]$ — a substring is a palindrome iff its outer characters match AND the inner substring (strictly shorter) is also a palindrome, correctly requiring the **shorter** interval $dp[i+1][j-1]$ to be already computed — which is why this DP must be filled by **increasing substring length**, not simple row-by-row order.

SECTION 5 — Algorithm Framework

5.1 General Template — Longest Common Subsequence

```
function lcs(A, B):
    m, n = length(A), length(B)
    dp = 2D array of size (m+1) x (n+1), all zeros

    for i in range(1, m+1):
        for j in range(1, n+1):
            if A[i-1] == B[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]
```

5.2 General Template — Edit Distance

```
function editDistance(A, B):
    m, n = length(A), length(B)
    dp = 2D array of size (m+1) x (n+1)

    for i in range(0, m+1): dp[i][0] = i
    for j in range(0, n+1): dp[0][j] = j

    for i in range(1, m+1):
        for j in range(1, n+1):
            if A[i-1] == B[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j],          # delete
                                   dp[i][j-1],          # insert
                                   dp[i-1][j-1])         # substitute

    return dp[m][n]
```

5.3 General Template — Longest Palindromic Substring (Interval DP)

```
function longestPalindromicSubstring(s):
    n = length(s)
    dp = 2D boolean array of size n x n, all false
    start, maxLen = 0, 1
    for i in range(0, n): dp[i][i] = true

    for length in range(2, n+1):
        for i in range(0, n - length + 1):
            j = i + length - 1
            if length == 2:
                dp[i][j] = (s[i] == s[j])
            else:
                dp[i][j] = (s[i] == s[j]) and dp[i+1][j-1]

            if dp[i][j] and length > maxLen:
```

```

start = i
maxLen = length

return s[start : start + maxLen]

```

5.4 Interview Thinking Process

1. "Am I comparing two distinct strings, or analyzing one string's internal structure?"
2. "For two-string comparison, I'll define $dp[i][j]$ over prefixes, with a match/mismatch branch in the recurrence."
3. "For single-string palindrome analysis, I'll use interval DP, filling by increasing substring length, since $dp[i][j]$ depends on the strictly shorter $dp[i+1][j-1]$."
4. "I'll carefully define base cases: empty-prefix comparisons for sequence DP, single/double-character substrings for palindrome DP."

SECTION 6 — Time & Space Complexity

Problem	Time	Space	Why
LCS	$O(m \times n)$	$O(m \times n)$, reducible to $O(\min(m,n))$	Standard 2D DP over both strings' prefixes
Edit Distance	$O(m \times n)$	$O(m \times n)$, reducible to $O(\min(m,n))$	Same structure as LCS, three-way min instead of two-way max
Longest Palindromic Substring (DP)	$O(n^2)$	$O(n^2)$	All $O(n^2)$ substrings potentially checked; can be reduced to $O(n)$ space with careful ordering, or $O(n)$ time with Manacher's algorithm (advanced)
Longest Palindromic Subsequence	$O(n^2)$	$O(n^2)$, reducible to $O(n)$	Same interval-DP structure as palindromic substring, but counts subsequence length not exact substring

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty string(s)	$A=""$ or $B=""$	LCS/Edit Distance base cases ($dp[i][0]/dp[0][j]$) directly handle this
Identical strings	$A=B="abc"$	LCS = length of the string; Edit Distance = 0

Edge Case	Example	Handling
Completely disjoint strings	No common characters at all	LCS = 0; Edit Distance = $\max(\text{len}(A), \text{len}(B))$ (all inserts/deletes)
Single-character string	A="a"	Trivial base case in both sequence and palindrome DP
Case sensitivity	"Cat" vs "cat"	Clarify whether comparison should be case-sensitive
Palindrome DP on an already-fully-palindromic string	s="aaaa"	Every substring is a palindrome — $\text{dp}[i][j]$ is true throughout, correctly identified
Very long strings (10^4+)	$O(n^2)$ time/space becomes significant	Consider space optimization or, for exact substring palindrome detection, Manacher's $O(n)$ algorithm

Common mistakes: off-by-one errors in the base-case initialization for empty-prefix comparisons; incorrect fill order for palindrome interval DP (must be by increasing length, not row-by-row, since $\text{dp}[i][j]$ depends on $\text{dp}[i+1][j-1]$, a "later row, earlier column" cell relative to naive row-major order).

SECTION 8 — Pros & Cons

Advantages: Correctly and efficiently solves a huge family of string-comparison and string-structure problems in $O(n^2)$ or $O(m \times n)$; recurrence design (match/mismatch branching) is a broadly reusable mental template. **Disadvantages:** $O(n^2)$ can be slow for very long strings (10^5+); interval DP's non-standard fill order (by length) is a common source of implementation bugs for those used to row-by-row grid DP. **Trade-offs:** Full DP table (supports reconstruction of the actual alignment/palindrome) vs. space-optimized rolling array (loses reconstruction ability without extra bookkeeping). **Limitations:** Doesn't scale well to very long strings without specialized algorithms (Manacher's for palindromes, suffix automata for advanced substring problems). **Inefficient when:** only existence (not longest/optimal) is needed — specialized string-matching algorithms (KMP, Z-algorithm, Rabin-Karp) are faster and more appropriate for pure existence/matching queries.

SECTION 9 — Real World Applications

Domain	Application
Version Control (Git/diff)	LCS-based diffing between file versions to show minimal changes
Bioinformatics	DNA/protein sequence alignment (LCS and Edit Distance are the foundational algorithms, extended with scoring matrices)
Spell Checkers	Edit Distance for "did you mean" suggestions and autocorrect
Plagiarism Detection	LCS/similarity scoring between documents

Domain	Application
Natural Language Processing	Machine translation evaluation metrics, text similarity scoring
Search Engines	Fuzzy/approximate string matching for typo-tolerant search
Data Deduplication	Edit-distance-based near-duplicate record detection
Text Editors	“Track changes” and diff visualization features
Genome Sequencing	Palindromic sequence detection (biologically significant DNA structures)
Compression Algorithms	Some dictionary-based compression schemes leverage longest-common-substring detection

SECTION 10 — Interview Strategy

How seniors answer: They immediately classify the problem as sequence-comparison (two strings) or interval/palindrome (one string), correctly derive the match/mismatch recurrence with justification (exchange argument for LCS’s match case), and recognize the non-standard fill order required for interval DP.

How juniors answer: They sometimes conflate the fill orders (attempting row-by-row for palindrome interval DP, which produces incorrect results since $dp[i+1][j-1]$ isn’t ready yet), or they get the base-case initialization for empty-prefix comparisons wrong.

Typical follow-ups: “Can you reconstruct the actual LCS/edit sequence, not just its length/count?” “Can you optimize space to $O(\min(m,n))$?” “How would Manacher’s algorithm solve the palindromic substring problem in $O(n)$ instead of $O(n^2)$?”

Optimization questions: “Can you do Edit Distance in $O(\min(m,n))$ space?” (Yes — same rolling-row technique as LCS, since the recurrence only references the previous row and current row.)

SECTION 11 — Pattern Variations

Variation	Description	Example
Longest Common Subsequence	Non-contiguous shared order	Longest Common Subsequence
Longest Common Substring	Contiguous shared sequence (different recurrence: resets to 0 on mismatch)	Longest Common Substring (GFG classic)
Edit Distance	Minimum insert/delete/substitute operations	Edit Distance
Longest Palindromic Substring	Contiguous palindrome	Longest Palindromic Substring
Longest Palindromic Subsequence	Non-contiguous palindrome	Longest Palindromic Subsequence
Minimum Palindrome Partitioning	Minimum cuts so every piece is a palindrome	Palindrome Partitioning II

Variation	Description	Example
Distinct Subsequences (Counting)	Count ways one string forms a subsequence of another	Distinct Subsequences
Interleaving String	Check if a string interleaves two others	Interleaving String
Wildcard/Regex Matching	Pattern matching with special characters	Wildcard Matching, Regular Expression Matching

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
2D Grid DP (general)	DP on Strings IS a specialized application of 2D DP	Recognize this as the same family, just string-specific
KMP/Rabin-Karp/Z-Algorithm	Pure existence/matching, no DP table, often $O(n+m)$	Only need to find if/where a pattern occurs, not longest/optimal comparison
Manacher's Algorithm	$O(n)$ specialized palindrome-substring algorithm	Need the absolute fastest palindromic substring solution, not the general DP approach
Suffix Trees/Arrays	Advanced structures for multi-pattern/multi-query substring problems	Need to answer many substring queries efficiently against a fixed string

Comparison Table

Aspect	Sequence-Comparison DP (LCS/Edit Distance)	Interval DP (Palindrome)	KMP/Rabin-Karp
Compares	Two distinct strings	One string against itself	Pattern existence in text
Fill order	Row-by-row (i, then j)	By increasing length	Linear scan
Time	$O(m \times n)$	$O(n^2)$	$O(n+m)$

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	N/A (rare at pure Easy for this family)	—
Medium	Basic LCS, basic palindrome substring	Longest Common Subsequence, Longest Palindromic Substring

Difficulty	Characteristics	Examples
Hard	Edit Distance, counting variants, three-way comparisons	Edit Distance, Distinct Subsequences, Interleaving String
Very Hard	Advanced pattern matching, minimum partitioning	Regular Expression Matching, Wildcard Matching, Palindrome Partitioning II

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Longest Common Subsequence	Medium	Amazon, Meta, Google, Microsoft	Direct sequence-comparison DP	Foundational recurrence mastery
2	Longest Common Substring (GFG classic)	Medium	Amazon, Google	Contiguous variant (reset-on-mismatch recurrence)	Contrast with LCS
3	Edit Distance	Hard	Amazon, Meta, Google, Microsoft	Three-way min recurrence	Advanced sequence-comparison DP
4	Longest Palindromic Substring	Medium	Amazon, Meta, Google, Microsoft	Interval DP (contiguous palindrome)	Foundational interval DP
5	Longest Palindromic Subsequence	Medium	Amazon, Google	Interval DP (non-contiguous palindrome)	Interval DP variant
6	Palindrome Partitioning II	Hard	Amazon, Google	Interval DP (palindrome check) + 1D DP (minimum cuts)	Cross-pattern combination
7	Palindrome Partitioning	Medium	Amazon, Meta	Interval DP + Backtracking (enumerate all partitions)	Cross-pattern (Interval DP + Backtracking)
8	Distinct Subsequences	Hard	Amazon, Google, Microsoft	Sequence-comparison counting DP	Counting-based sequence DP

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
9	Interleaving String	Medium	Amazon, Google	Three-sequence comparison collapsing to 2D	Advanced sequence DP
10	Regular Expression Matching	Hard	Amazon, Meta, Google, Microsoft	Sequence DP with pattern-matching recurrence	Advanced pattern-matching DP
11	Wildcard Matching	Hard	Amazon, Google, Microsoft	Sequence DP with wildcard handling	Advanced pattern-matching DP variant
12	Shortest Common Supersequence	Hard	Google, Amazon	LCS-based construction + reconstruction	DP + reconstruction technique
13	Delete Operation for Two Strings	Medium	Amazon, Google	LCS-based reduction	Problem reduction to LCS
14	Minimum ASCII Delete Sum for Two Strings	Medium	Amazon, Google	Cost-weighted LCS variant	Weighted sequence DP
15	Uncrossed Lines	Medium	Amazon, Google	Direct LCS equivalence	Problem-recognition mastery
16	Is Subsequence	Easy	Amazon, Meta, Adobe	Simple two-pointer or DP-based subsequence check	Foundational contrast (simpler technique suffices)
17	Count Different Palindromic Subsequences	Hard	Google	Advanced interval DP with distinct-counting	Advanced interval DP variant
18	Longest Repeating Substring (contrast, suffix-based)	Medium	Google	Contrast: suffix array/binary search, related family	Pattern-boundary awareness
19	Valid Palindrome II (contrast, two pointers)	Easy	Amazon, Meta	Contrast: Two Pointers with one-deletion allowance, not full DP	Pattern-boundary awareness
20	Palindromic Substrings (Count)	Medium	Amazon, Meta	Interval DP or expand-around-center	Counting variant of interval DP

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
21	Longest Palindromic Substring (expand-around-center contrast)	Medium	Amazon, Meta	Contrast: $O(n^2)$ time, $O(1)$ space alternative to DP	Technique trade-off awareness
22	Minimum Insertion Steps to Make a String Palindrome	Hard	Amazon, Google	LCS-with-reverse or Edit-Distance-style reduction	Problem reduction mastery
23	Shortest Palindrome (contrast, KMP-based)	Hard	Google, Amazon	Contrast: KMP-based approach, not pure DP	Pattern-boundary awareness
24	Scramble String	Hard	Google, Amazon	Advanced interval + recursive partition DP	Advanced recursive interval DP
25	Longest Common Subsequence of Three Strings (variant)	Hard	Google	3D DP extension of LCS	Dimensional extension awareness
26	Number of Ways to Form a Target String Given a Dictionary	Hard	Google	Sequence-position + counting hybrid DP	Advanced hybrid DP
27	Longest Duplicate Substring (contrast, binary search + hashing)	Hard	Google	Contrast: Binary Search + Rolling Hash, not pure DP	Pattern-boundary awareness
28	K-Similar Strings (contrast, BFS)	Hard	Google	Contrast: BFS over permutation states, not DP	Pattern-boundary awareness
29	One Edit Distance (simplified variant)	Medium	Amazon, Meta	Simplified single-pass check (contrast with full Edit Distance DP)	Simplified-case recognition

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
30	Text Justification (contrast, greedy/1D DP)	Hard	Amazon, Meta	Contrast: 1D DP for line-break optimization, not string-comparison DP	Pattern-boundary awareness

SECTION 15 — Common Mistakes

1. Using row-by-row fill order for palindrome interval DP, when it must be filled by increasing substring length (since $dp[i][j]$ depends on $dp[i+1][j-1]$, which isn't ready in naive row-major order). *Fix*: always iterate by length for interval DP.
2. Incorrect base-case initialization for empty-prefix comparisons in LCS/Edit Distance ($dp[i][0]/dp[0][j]$). *Fix*: explicitly reason through what comparing against an empty string means for the specific problem.
3. Confusing Longest Common Subsequence (non-contiguous) with Longest Common Substring (contiguous) — these have different recurrences (LCS's mismatch case takes $\max(dp[i-1][j], dp[i][j-1])$; Substring's mismatch case resets to 0). *Fix*: clarify which variant is being asked before coding.
4. Forgetting that Edit Distance's match case is $dp[i-1][j-1]$ (no operation needed), not $dp[i-1][j-1]+1$ — accidentally adding an unnecessary operation cost even when characters already match. *Fix*: carefully distinguish match (no cost) from mismatch (cost 1) branches.
5. Not recognizing when a problem reduces to a known variant (e.g., "Delete Operation for Two Strings" reduces directly to LCS: $\text{answer} = (m - \text{lcs}) + (n - \text{lcs})$). *Fix*: always check whether the target problem is a disguised/transformed version of LCS, Edit Distance, or palindrome DP.

Why people fail: the recurrences for LCS, Edit Distance, and palindrome DP look superficially similar (all are 2D tables with match/mismatch branching), but subtle differences in the mismatch-case combination (max vs. min vs. reset-to-zero) and fill order (row-by-row vs. by-length) are easy to conflate, especially under time pressure when working from memory rather than first principles.

SECTION 16 — Optimization Techniques

- **Time:** Recognize disguised reductions to already-known problems (e.g., "Delete Operation for Two Strings" $\rightarrow$ LCS) to avoid re-deriving a new recurrence from scratch; use Manacher's $O(n)$ algorithm instead of $O(n^2)$ DP for pure longest-palindromic-substring queries when performance is critical.
- **Space:** Space-optimize LCS and Edit Distance from $O(m \times n)$ to $O(\min(m, n))$ using a rolling-row technique, since both recurrences only reference the immediately previous row.
- **Readability:** Clearly comment the match vs. mismatch branches and their distinct combination logic (max/min/reset) for each specific problem variant.
- **Interview performance:** Explicitly classify the problem as sequence-comparison or interval/palindrome DP before coding, and state the fill-order requirement (row-by-row vs. by-length) proactively for interval DP problems.

SECTION 17 — Multiple Language Templates

Java

```
public int longestCommonSubsequence(String text1, String text2) {
    int m = text1.length(), n = text2.length();
    int[][] dp = new int[m+1][n+1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i-1) == text2.charAt(j-1)) dp[i][j] = dp[i-1][j-1] + 1;
            else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[m][n];
}
```

JavaScript

```
function longestCommonSubsequence(text1, text2) {
    const m = text1.length, n = text2.length;
    const dp = Array.from({length: m+1}, () => new Array(n+1).fill(0));
    for (let i = 1; i <= m; i++) {
        for (let j = 1; j <= n; j++) {
            if (text1[i-1] === text2[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
            else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[m][n];
}
```

PHP

```
function longestCommonSubsequence(string $text1, string $text2): int {
    $m = strlen($text1); $n = strlen($text2);
    $dp = array_fill(0, $m + 1, array_fill(0, $n + 1, 0));
    for ($i = 1; $i <= $m; $i++) {
        for ($j = 1; $j <= $n; $j++) {
            if ($text1[$i-1] === $text2[$j-1]) $dp[$i][$j] = $dp[$i-1][$j-1] + 1;
            else $dp[$i][$j] = max($dp[$i-1][$j], $dp[$i][$j-1]);
        }
    }
    return $dp[$m][$n];
}
```

Python

```
def longest_common_subsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
```

```

        dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

```

Go

```

func longestCommonSubsequence(text1 string, text2 string) int {
    m, n := len(text1), len(text2)
    dp := make([][]int, m+1)
    for i := range dp {
        dp[i] = make([]int, n+1)
    }
    for i := 1; i <= m; i++ {
        for j := 1; j <= n; j++ {
            if text1[i-1] == text2[j-1] {
                dp[i][j] = dp[i-1][j-1] + 1
            } else if dp[i-1][j] > dp[i][j-1] {
                dp[i][j] = dp[i-1][j]
            } else {
                dp[i][j] = dp[i][j-1]
            }
        }
    }
    return dp[m][n]
}

```

C++

```

int longestCommonSubsequence(string text1, string text2) {
    int m = text1.size(), n = text2.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1[i-1] == text2[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
            else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[m][n];
}

```

SECTION 18 — Dry Runs

Small Input

text1="abcde", text2="ace" (LCS)

		a	c	e
	0	0	0	0
a	0	1	1	1
b	0	1	1	1
c	0	1	2	2
d	0	1	2	2
e	0	1	2	3

`dp[5][3] = 3 → LCS = "ace", length 3`

Large Input (Conceptual)

For two strings of length 1000 each, LCS/Edit Distance costs $O(10^6)$ operations — fast and feasible, versus the exponential number of possible alignments (combinatorially infeasible to brute-force directly).

Corner Case

`text1="", text2="abc": dp[0][j] = 0 for all j` (empty string has no common subsequence with anything) → `dp[0][3] = 0`, correctly representing zero-length LCS.

SECTION 19 — Advanced Concepts

- **Hirschberg's Algorithm:** achieves LCS/Edit Distance reconstruction (the actual alignment, not just its length) in $O(m \times n)$ time but only $O(\min(m, n))$ space, using a divide-and-conquer approach combined with the space-optimized DP — a sophisticated technique worth mentioning for Staff-level space-optimization discussions.
 - **Manacher's Algorithm:** solves Longest Palindromic Substring in $O(n)$ time (versus $O(n^2)$ DP or $O(n^2)$ expand-around-center), using a clever exploitation of previously-computed palindrome radii to avoid redundant character comparisons — an advanced, highly specialized algorithm worth knowing exists even if rarely required to implement from scratch.
 - **Problem transformation chains:** many “hard” string DP problems are transformations of LCS or Edit Distance in disguise — “Delete Operation for Two Strings” reduces to LCS via $(m - \text{lcs}) + (n - \text{lcs})$; “Minimum Insertion Steps to Make a String Palindrome” reduces to $n - \text{LCS}(s, \text{reverse}(s))$; recognizing these reductions turns seemingly novel Hard problems into applications of already-mastered recurrences.
 - **Scramble String / recursive interval partitioning:** some string DP problems require considering all possible ways to **split** a substring into two parts and recursively check both (with an optional swap), combining interval DP with an additional partition-point search — a more advanced generalization of the basic interval DP template.
-

SECTION 20 — Senior Engineer Insights

Staff engineers recognize DP on Strings as the algorithmic foundation of **diff tools, spell checkers, and bioinformatics sequence alignment** — genuinely high-value, widely-deployed production systems. They're fluent in recognizing problem transformations (reducing seemingly novel problems to LCS/Edit Distance/palindrome DP), know when specialized faster algorithms exist (Manacher's for palindromes, Hirschberg's for space-efficient reconstruction), and correctly distinguish sequence-comparison DP's row-by-row fill order from interval DP's by-length fill order — a distinction that trips up even experienced engineers who haven't internalized *why* the orders differ (dependency direction). Interviewers evaluate this precise understanding, not just pattern-matched code production.

SECTION 21 — Cheat Sheet

PATTERN: DP on Strings (LCS, Edit Distance, Palindromes)

RECOGNIZE: "longest common subsequence/substring," "edit distance," "minimum operations to convert," "pal."

TEMPLATE (LCS):

`dp[i][j] = dp[i-1][j-1]+1 if A[i-1]==B[j-1] else max(dp[i-1][j], dp[i][j-1])`

TEMPLATE (Edit Distance):

$dp[i][j] = dp[i-1][j-1]$ if match else $1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$

TEMPLATE (Palindrome interval DP):

$dp[i][j] = (s[i] == s[j])$ and $dp[i+1][j-1]$ - FILL BY INCREASING LENGTH

COMPLEXITY: $O(m \times n)$ sequence comparison; $O(n^2)$ palindrome interval DP

KEY PROOF: optimal substructure via exchange argument (LCS match case) or exhaustive last-operation case

WATCH FOR: row-by-row (sequence DP) vs by-length (interval DP) fill order, base-case initialization, match

DOESN'T APPLY WHEN: only existence/matching needed (use KMP/Rabin-Karp), extremely long strings needing O

SECTION 22 — Revision Notes (5-Minute Review)

- Sequence-comparison DP (LCS, Edit Distance): $dp[i][j]$ over prefixes of two strings, row-by-row fill.
 - LCS match case: $dp[i-1][j-1] + 1$; mismatch: $\max(dp[i-1][j], dp[i][j-1])$.
 - Edit Distance match case: $dp[i-1][j-1]$ (no cost); mismatch: $1 + \min(\text{delete}, \text{insert}, \text{substitute})$.
 - Interval DP (palindromes): $dp[i][j]$ over a substring range, filled by INCREASING LENGTH, not row-by-row.
 - Many “hard” problems reduce to LCS/Edit Distance/palindrome DP via transformation — always check for this first.
 - Manacher’s algorithm: $O(n)$ for longest palindromic substring, an advanced alternative to $O(n^2)$ DP.
-

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic LCS and palindrome substring	Longest Common Subsequence (1143), Longest Palindromic Substring (5)
Intermediate	Edit Distance, palindrome subsequence	Edit Distance (72), Longest Palindromic Subsequence (516)
Advanced	Counting variants, partitioning	Distinct Subsequences (115), Palindrome Partitioning II (132), Interleaving String (97)
Expert	Advanced pattern matching, reconstruction	Regular Expression Matching (10), Wildcard Matching (44), Shortest Common Supersequence (1092)

SECTION 24 — Memory Tricks

- **Mnemonic:** “**M**atch **D**iagonal, **M**ismatch **A**bove/**L**eft” (for sequence DP); “**O**uter Match + **I**nnner Palindrome” (for interval DP).

- **Visualization: Aligning two strips of paper** to find matching letters in order (LCS); **peeling matching letters from both ends inward** (palindrome check).
 - **Recognition shortcut:** Two strings compared → sequence DP, row-by-row. One string, symmetry/palindrome → interval DP, by-length.
-

SECTION 25 — Final Summary

DP on Strings applies the general 2D DP principle specifically to sequence comparison (LCS, Edit Distance — two strings, row-by-row fill) and palindrome/interval analysis (one string, by-length fill), both built from provably correct match/mismatch recurrences. The single most important thing to remember forever: **sequence-comparison DP fills row-by-row because it depends on the immediately previous row, while palindrome interval DP MUST fill by increasing substring length because $dp[i][j]$ depends on the strictly shorter, inner interval $dp[i+1][j-1]$ — conflating these two fill orders is the most common structural bug in this pattern family.** Many seemingly novel “hard” string problems are disguised transformations of LCS, Edit Distance, or palindrome DP — always check for this reduction before deriving a new recurrence from scratch.